

Guión de presentación

TP3 — Perceptrones

Orden cronológico de la presentación

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Cómo está organizado este guion

Este documento sigue el **orden real de la presentación**, slide por slide. / Cada bloque tiene un orador asignado — los oradores se cruzan varias veces. / Si lo lees de arriba hacia abajo en el momento de presentar, sabés exactamente dónde estás y quién habla.

Secuencia (35 slides totales, 24 minutos):

#	Bloque	Tema	Slides	Tiempo
1	A — Mateo	Apertura + Ej 1 setup + dataset	1–5	2:55
2	B — Katia	Lineal vs no-lineal + threshold + recomendación	6–8	3:35
3	C — Nicolás	Setup Ej 2 + fases + sweeps arch/opt	9–13	3:30
4	D — Tomás	Decisiones de diseño	14–16	3:35
5	E — Nicolás	Learning rate + convergencia/bias	17–20	3:15
6	F — Tomás	Curvas + matriz + final eval (cachetazo)	21–24	2:50
7	G — Mateo	Hipótesis Ej 3 + el dominó +10 puntos	26–27	1:55
8	H — Katia	Matriz ganador + tabla resultados + comparación	28–30	1:45
9	I — Mateo	Curvas del ganador + qué ayudó	31–32	2:00
10	J — Katia	Por qué no 98 % + conclusiones	33–34	2:10

Las slides 9 (dividir Ej 2), 25 (dividir Ej 3) y 35 (Gracias) las cubre el orador adyacente, sin contar tiempo.

Totales por orador (sumando sus bloques):

Persona	Bloques	# slides	Tiempo
Mateo	A + G + I	9	6:50
Katia	B + H + J	8	7:30
Nicolás	C + E	8	6:45
Tomás	D + F	7	6:25

Reglas no negociables:

- **Se lee como uno habla** — español rioplatense, fluido, sin tecnicismos innecesarios. Si lo lees tal cual, te sale natural.

- **No se dice “distribution shift”.** Se dice “los datos no se parecen”, “el test viene de otra población”, “estilos de escritura distintos”.
- **No se dice “Pack C”.** Esa nomenclatura quedó atrás; ahora hablamos de “regularización”.

Convenciones: 0:30 = dónde tendrías que estar dentro del bloque · [Slide N] = cambiar de slide · / pausa breve · // pausa más larga · → **Mateo**→**Katia** = pasarle el bastón.

Índice

1. Bloque A / Mateo / Apertura + Ej 1 setup	1
2. Bloque B / Katia / Lineal vs no-lineal + threshold	2
3. Bloque C / Nicolás / Setup Ej 2 + sweeps arch/opt	2
4. Bloque D / Tomás / Decisiones de diseño	3
5. Bloque E / Nicolás / Learning rate + convergencia/bias	4
6. Bloque F / Tomás / Curvas, matriz y final eval	5
7. Bloque G / Mateo / Hipótesis Ej 3 + el dominó	6
8. Bloque H / Katia / Matriz, tabla y comparación visual	7
9. Bloque I / Mateo / Curvas del ganador + qué ayudó	7
10. Bloque J / Katia / Por qué no 98 % + conclusiones	8
Apéndice — preguntas anticipadas	8

Bloque A / Mateo / Apertura + Ej 1 setup

/

/ 2:55

Foco: arrancar la presentación (portada + setup del Ejercicio 1 + análisis del dataset). Es el escenario — todavía no hay datos fuertes.

Q&A: si te preguntan algo del dataset → derivá a Katia. De regularización → Tomás.

[Slide 1, portada] [Bloque A]

Buenas. Trabajo Práctico tres, perceptrones. / Somos cuatro: Katia, Nicolás, Tomás y yo. Nos vamos turnando en bloques cortos, así que vamos a estar yendo y viniendo. / Al final dejamos lugar para preguntas.

0:25

[Slide 2, divider Ej 1]

Arrancamos con el Ejercicio uno, que es lo que llaman *knowledge distillation* sobre un dataset de fraude. / El dos y el tres son de clasificación de dígitos manuscritos, los vemos después.

0:45

[Slide 3, Ej 1 — el problema]

A ver, qué nos pide el Ejercicio uno. // Tenemos un BigModel que ya viene entrenado y clasifica fraude. Para nosotros es una caja negra: lo único que vemos es su salida, una probabilidad continua entre cero y uno. / La idea es entrenar un modelo chico — le decimos *TinyModel* — que sea un perceptrón simple y replique esa salida.

// El dataset tiene nueve features de la transacción: monto, duración de sesión, edad de la cuenta, esas cosas. / Y trae también una columna que se llama `flagged_fraud`, que es la etiqueta binaria de fraude real. *El enunciado dice claramente que esa columna no se puede usar para entrenar*: la usamos solo después, para sacar las métricas operativas — precision, recall, F1.

// Como la salida tiene que estar entre cero y uno, la activación final del perceptrón es **sigmoide**.

1:30

[Slide 4, Ej 1 — análisis del dataset]

Antes de entrenar nada, miramos el dataset y nos encontramos con algo bastante llamativo. // Hay tres umbrales bien marcados, en tres features distintas, que separan las clases casi perfecto. / Si el monto es mayor a 500, o la cantidad comprada es diez o más, o los items vistos antes de la compra son quince o más / *cualquiera de las tres*, casi seguro es fraude.

// La tabla de la izquierda muestra cuántos casos cubre cada regla por separado, y cuántos falsos positivos genera: **cero falsos positivos en las tres**. / Y la de la derecha junta las tres con un OR lógico: precision 100 %, recall 80 %.

2:00

[Slide 5, Ej 1 — conclusión del análisis]

Lo que nos quedó de eso, que está acá en la caja del medio: // **el 80 % del fraude lo agarrás con tres ifs y no te queda un solo falso positivo**. / El 20 % que falta — 174 casos — son los fraudes “sutiles”: los que respetan los tres umbrales y solo se detectan combinando varias features.

// Y eso nos dejó tres conclusiones para entrenar el TinyModel. / Primero, el modelo *tiene* que igualar las tres reglas — recall arriba del 80 % es el piso. / Segundo, lo que aporta el modelo se mide en cuántos de esos 174 casos sutiles logra recuperar. / Y tercero, los saltos duros que vemos — de 6 % a 100 % — son imposibles para un modelo lineal. Acá el perceptrón *no lineal* debería ganarle clarito al lineal.

→ **Mateo→Katia** / Le paso a Katia para que cuente cómo se midió esa diferencia.

Bloque B / Katia / Lineal vs no-lineal + threshold

/ / 3:35

Foco: cerrar el Ejercicio 1 con la decisión de threshold (lo más de negocio del TP).

Q&A: threshold y trade-offs → vos.

[Slide 6, Ej 1 — Lineal vs No-Linear] [Bloque B]

Buenas, soy Katia. / Mateo dejó la pregunta abierta de si el perceptrón *no lineal* le ganaba al lineal. La respuesta corta es que sí, y bastante.

// Acá, en la tabla del medio, comparamos las dos versiones. Mismo dataset, mismo K-fold=5 estratificado, mismas features. / Lo único que cambia entre los dos es la sigmoide. // El *lineal* —que es Adaline puro— da MSE de 0.0266. / El *no lineal*: **0.0110. 59 % menos de error**.

// ¿Por qué nos da una diferencia tan grande? / Acordate que el target es una probabilidad: tiene que estar entre cero y uno. / La sigmoide *automáticamente* mapea su salida a ese rango. El lineal puede tirarte valores fuera de cero a uno, y el MSE te castiga fuerte cuando la salida queda fuera del dominio. / No es magia, es que la activación final tiene que ser coherente con la naturaleza del target.

1:10

[Slide 7, Ej 1 — pruebas sobre threshold]

Ahora viene la parte que más nos hizo pensar. // Si tomamos el modelo no lineal con threshold 0.5 —el default obvio para una sigmoide— las métricas operativas dan mal. / Precision 33 %, recall 100 %. Tira casi todo a fraude.

// A primera vista parece un modelo malísimo. Pero la cuestión es que el modelo no es el problema — *el threshold no está calibrado al modelo*. / ¿Por qué? Porque las salidas de la sigmoide se concentran en la parte de arriba. La mediana de los scores de las muestras de fraude está en 0.99. Y los *no fraudes* no se van a 0.05 como uno esperaría — quedan en 0.4 o 0.6. / La red aprendió un *ranking* muy bueno, pero no un calibrado perfecto.

// La solución fue barrer el threshold de cero a uno con paso 0.01. En el plot ven precision en azul, recall en verde, F1 en negro. / El óptimo por F1 está en **threshold 0.89**: precision 89, recall 86, F1 87, accuracy 97 %.

2:10

[Slide 8, Ej 1 — recomendación de umbral]

Y la última pregunta del Ejercicio uno: *qué umbral le recomendamos al cliente*. // Y la respuesta es: depende. Depende del costo asimétrico que tenga el cliente entre falso positivo y falso negativo.

// La tabla muestra cuatro puntos. / Threshold 0.89 maximiza F1: agarra el 86 % de los fraudes con 96 falsos positivos. Esto sirve si auditar manualmente cada flagged es barato. / Threshold 0.95 baja a 35 falsos positivos pero pierde recall: solo recupera el 75 %. Esto sirve si los falsos positivos son caros para el negocio. / Y threshold 0.99 te da precision perfecta y cero falsos positivos — pero pierde la mitad de los fraudes reales. Esto es para cuando hay tolerancia cero a falsos positivos.

// El modelo entrega scores. Decidir el umbral es una decisión del negocio.

→ **Katia**→**Nicolás** Le paso a Nicolás para arrancar el Ejercicio dos.

Bloque C / Nicolás / Setup Ej 2 + sweeps arch/opt

/ / 3:30

Foco: arrancar el Ejercicio 2 (problema, fases del experimento, sweep arquitectura y optimizador). Tablas y gráficos hablan solos.

Q&A: datos / config → vos. Si profundizan en por qué Adam → Tomás.

[Slide 9, divider Ej 2] [Bloque C]

Buenas, soy Nicolás. / Pasamos al Ejercicio dos.

0:15

[Slide 10, Ej 2 — el problema]

Clasificación multiclase de dígitos manuscritos del cero al nueve. // Cada muestra es una imagen de veintiocho por veintiocho píxeles, en escala de grises, aplanada en *un vector de 784 floats entre cero y uno*. / Train: 12.449 muestras. Test: 2.498.

// La arquitectura final, después de todos los sweeps que les voy a mostrar, es: / entrada de 784, capa oculta de 100 con ReLU, otra de 50 con ReLU, salida de 10 con softmax. / Y la regla del juego, en la caja roja: **el test se usa una sola vez, al final**. La búsqueda de hiperparámetros es solo sobre `digits.csv`.

1:00

[Slide 11, Ej 2 — fases de experimentación]

Para no probar al azar, dividimos la búsqueda en **dos fases**. // *Fase uno, exploratoria*: K-folds = 1, o sea un solo train-val ochenta-veinte. / Sweeps secuenciales: arquitectura, después optimizador, después learning rate, después batch size. / Cada sub-fase usa el ganador de la anterior, así reducimos un montón el espacio de búsqueda.

// *Fase dos, validación*: K-fold = 5. / Tomamos el config ganador de la primera fase y variamos un solo hiperparámetro a la vez para ver si las elecciones se sostienen con K-fold completo. / Y al final, un único *final eval* contra el test set — una sola vez, como dije recién.

1:40

[Slide 12, Ej 2 — sweep de arquitectura]

Empezamos por arquitectura. // Probamos cinco variantes alrededor del ganador. / La fila verde es el ganador: 784, 100, 50, 10. Validación 96.74, y 96.22 con K-fold=5. / La de abajo, 128-64, fue el empate técnico más cercano: 0.04 arriba pero *dentro del desvío*. // La que dice *wider*, 200-100, ganó 0.17 en validación pero perdió 0.48 en test, y tardó 3.5 veces más en entrenar. / *Deeper* empató, y *shallow* con 30 neuronas en una sola capa quedó subajustada.

// La decisión la tomamos por *parsimonia* —regla de Occam—: cuando dos modelos rinden igual, te quedás con el más simple y rápido. / Y el experimento de *wider* confirmó la curva U que vimos en la clase de regularización: si agrandás la red sin compensar con más datos, te metés en zona de overfitting.

2:30

[Slide 13, Ej 2 — sweep de optimizador]

Sweep de optimizador. / **Adam** con learning rate 0.001: 96.74, *best epoch* 7. Ganador. / Momentum con $\beta = 0,9$: 96.34, converge más lento —*best epoch* 18. / SGD vanilla: 94.81, y la realidad es que *no convergió en 50 epochs*, llegó al límite todavía mejorando. // Adam ganó por convergencia más rápida y resultado igual o mejor. Momentum quedó dentro del desvío, SGD sin aceleración no puede.

→ **Nicolás→Tomás** Le paso a Tomás para que cuente *por qué* elegimos cada cosa — las decisiones de diseño con backing teórico.

Bloque D / Tomás / Decisiones de diseño

/ / 3:35

Foco: las decisiones donde el Q&A puede ir más profundo (activación, init, optimizador). El bloque más denso técnicamente.

Q&A: vanishing gradient, He vs Xavier, defaults de Adam → vos.

[Slide 14, Función de activación] [Bloque D]

Buenas, soy Tomás. / Nicolás mostró *qué* elegimos en cada sweep, ahora me toca contar el *por qué*. // Tres decisiones de diseño que aplican a todos los ejercicios: activación, inicialización y optimizador. / El learning rate y la función de costo los retoma Nicolás cuando vuelva.

// Función de activación, caso por caso. / En el Ejercicio uno usamos **sigmoide** porque el target es una probabilidad acotada en cero a uno. / En Ejercicio dos y tres, las capas ocultas con **ReLU** y la salida con **softmax**. // ¿Por qué ReLU en las capas ocultas? Porque el gradiente es fuerte y constante en zona positiva. *No satura* como tanh o sigmoide. / Y eso me lleva a la pregunta más importante del slide: **¿qué quiere decir saturar?** / Una activación *satura* cuando su pendiente se acerca a cero para entradas grandes en valor absoluto. Sigmoide y tanh saturan en sus extremos: el gradiente que llega hacia atrás se desvanece — vanishing gradient — y la red deja de aprender. / ReLU en su zona activa tiene gradiente constante $e = 1$: el gradiente fluye sin atenuarse. Por eso ReLU es la activación estándar moderna para capas ocultas. // Y softmax en la salida es directo: para clasificar 10 clases, queremos una distribución de probabilidad sobre las diez. Softmax la devuelve normalizada.

1:15

[Slide 15, Inicialización de pesos]

Inicialización. // *La regla de la cátedra*, que sale del HTML que nos compartieron, es: *valores chicos, no cero, no muy grandes*. / ¿Por qué no cero? Por simetría. Si todas las neuronas arrancan iguales, hacen exactamente lo mismo en cada paso del gradiente y *nunca se diferencian*. / ¿Por qué chicos? Porque pesos grandes saturan las activaciones — otra vez vanishing gradient — la salida queda en zonas planas, el gradiente es casi cero, la red no aprende.

// Tres esquemas. **He**: distribución normal con desvío $\sqrt{2/\text{fan_in}}$, para capas con ReLU. La intuición es que ReLU descarta la mitad del input, así que doblamos la varianza para compensar. / **Xavier**: $\sqrt{1/\text{fan_in}}$, para tanh, sigmoide y softmax, que son simétricas alrededor de cero. / **Uniforme** entre menos 0.1 y más 0.1, para el perceptrón simple del Ejercicio uno donde no hay propagación profunda.

2:15

[Slide 16, Optimizador]

Optimizador. // *La clase introduce gradiente descendente clásico*; Momentum y Adam son extensiones modernas. Comparamos los tres. // **SGD vanilla**: la versión mínima, w menos eta por gradiente. Funciona pero es lento, sobre todo en superficies con valles alargados. / **Momentum** con $\beta = 0,9$: acumula *velocidad* en la dirección del gradiente. Atraviesa valles sin oscilar. / **Adam**, con sus tres hiperparámetros default: combina *momentum* con *learning rate adaptativo por parámetro*. / Cada peso se ajusta con su propia escala según cuánto varía históricamente.

// **Resultado del sweep en el Ejercicio dos:** Adam 96.74 con best epoch 7. Momentum 96.34 — empate técnico pero converge más lento. SGD 94.81 — no convergió en 50 epochs. // Elegimos Adam por convergencia más rápida y resultado igual o mejor.

→ **Tomás→Nicolás** Le paso a Nicolás para learning rate y los detalles de convergencia y bias.

Bloque E / Nicolás / Learning rate + convergencia/bias

/ / 3:15

Foco: sweep de learning rate (con extremos catastróficos) y dos detalles de implementación (convergencia y bias).

Q&A: si profundizan en learning rate adaptativo → Tomás.

[Slide 17, Learning rate — cuál y por qué] [Bloque E]

Vuelvo yo. // Tomás cubrió las decisiones de diseño; ahora me toca contar cómo encontramos el learning rate ganador. // *La recomendación de la clase es: probar varios órdenes de magnitud.* / Los riesgos son simples: si lo elegís muy grande, la red oscila o diverge; si lo elegís muy chico, no converge en tiempo razonable.

// La búsqueda fue en **dos pasos**, igual que el resto. / Fase uno exploratoria, con cinco valores entre 10^{-4} y 10^{-2} . / Fase dos con esos mismos cinco más *extremos catastróficos* — 0.1 y diez — para mapear el rango donde diverge.

0:50

[Slide 18, learning rate — Fase 1]

Fase uno con K-folds = 1. // La fila verde, learning rate 0.001: 96.74, best epoch 7. Ganador. / 0.0005 queda muy cerca pero converge más lento, best epoch 17. / 0.0001 no termina de converger en 50 epochs. / Y 0.005 con 0.01 hacen overshoot —best epoch 2 a 4, demasiado temprano.

// Tres criterios apuntan al mismo valor: máximo de validación, convergencia rápida, sin overshoot.

1:30

[Slide 19, learning rate — Fase 2]

Y la confirmación con K-fold=5. // Mismos cinco valores más los extremos. / 0.001: 96.22, ganador. / 0.0001: 3 veces más lento, val_acc apenas peor. / 0.01: overshoot. // Y los extremos: 0.1 cae a 29% — divergencia clara. / Y **learning rate igual a diez** deja a la red en 12.48 — accuracy random, *equivalente a tirar una moneda con diez caras*. El loss explotó a NaN antes de converger.

// Conclusión: **learning rate 10^{-3} confirmado por dos sweeps independientes**. Robusto frente a la varianza del K-fold.

2:15

[Slide 20, convergencia y bias]

Y para terminar mi parte, dos detalles de implementación que no son hiperparámetros pero *el TP nos pide discutir*. // **Criterio de convergencia.** / En el Ejercicio uno usamos un *epsilon absoluto* sobre el MSE de train — la regla del apunte para perceptrón simple. / En Ejercicio dos y tres usamos **early stopping sobre el loss de validación** con patience de 10 epochs. / La razón: si miramos solo el train, no detectamos overfitting. Con validación, sí.

// **Cómo manejamos el bias.** / Truco del bias: a cada capa le agregamos una columna de unos al input antes del producto matricial, y los pesos pasan a tener shape (**n_out, n_in+1**). / El bias queda como un peso más y se actualiza con la misma regla del gradiente.

→ **Nicolás→Tomás** Le paso a Tomás para las curvas, la matriz y el cachetazo del final eval.

Bloque F / Tomás / Curvas, matriz y final eval

/ / 2:50

Foco: la parte clave del Ejercicio 2 — curvas, matriz y el momento más fuerte de la presentación: el cachetazo del final eval, donde nace toda la motivación del Ejercicio 3.

Tono: narrativo. Construí el cachetazo con la pausa correcta antes de decir “86.30”.

Q&A: cambio de población → vos.

[Slide 21, Ej 2 — curvas del modelo base] [Bloque F]

Vuelvo yo. // Estas son las curvas de aprendizaje del modelo base, evaluadas con K-fold=5. / Train y validación bajan parejas, convergencia limpia en alrededor de 5 epochs. / **Sin overfitting visible:** las dos curvas se mantienen pegadas hasta el final.

0:25

[Slide 22, Ej 2 — matriz del base]

Matriz de confusión sobre validación. // Diagonal limpia para 9 de las 10 clases. / La excepción —y esto es clave— es la **clase 8**: precision cero, recall cero. La red *nunca predice ocho*. / Eso ya nos dio una pista de que algo raro pasaba con la distribución de los datos. Quédense con esto, va a volver al final.

0:55

[Slide 23, Ej 2 — final eval: el cachetazo]

Y entonces hicimos el *final eval*. / Entrenamos con todo `digits.csv` y evaluamos una sola vez contra el test set, `digits_test.csv`. // Resultado.

// Validación K-fold=5: **96.22 %**. / Test sobre el set de prueba: **86.30 %**. // **10 puntos de diferencia.**

// Eso fue una sorpresa fuerte, sinceramente. / Diez veces más grande que el desvío estándar del K-fold —que era 0.43 %—, así que no se explica por azar. Hay algo más.

// La caja roja lo dice corto: *cambio de población, no underfitting*. El modelo está bien, los datos no se parecen.

1:35

[Slide 24, Ej 2 — matriz sobre el test]

Y la matriz de confusión sobre el test lo confirma. // La diagonal sigue siendo dominante, pero fíjense que las confusiones se reparten por todo el grid: no se acumulan en una sola clase. / **Errores distribuidos, no concentrados.** / Cuando los errores se concentran, suele ser falta de capacidad o falta de cobertura puntual. Cuando se reparten parejo, suele ser que *el train y el test son poblaciones distintas*.

// La interpretación es entonces: el set de prueba tiene variaciones —estilos de escritura, grosor de trazo, rotación, traslación— que en el train están sub-representadas. / Si esto es cierto, lo que hace falta no es un modelo más grande: hace falta **más datos que cubran lo que falta**.

→ **Tomás→Mateo** Le paso a Mateo, que cuenta justo cómo el Ejercicio tres confirmó esa hipótesis con un solo cambio de config.

Bloque G / Mateo / Hipótesis Ej 3 + el dominó

/ / 1:55

Foco: el momento “mirá esto” — los datos hablan solos. Más datos = +10 puntos en test, sin tocar el modelo.

[Slide 25, divider Ej 3] / (Lo cubrís de paso, sin contar tiempo.)

[Slide 26, Ej 3 — hipótesis y plan] [Bloque G]

Vuelvo yo. // Tomás recién mostró que el modelo del Ejercicio dos rendía 96 en validación pero se caía a 86 en el test. 10 puntos de diferencia. / Eso no se explica por azar.

// Entonces la hipótesis con la que arrancamos el Ejercicio tres fue esta: el problema no es el modelo — el modelo está bien. El tema es que *el test viene de una población distinta a la del train*. / Si la hipótesis es esa, lo primero que hay que probar es *más datos* que cubran lo que falta, no un modelo más grande.

// *Y el enunciado del Ejercicio tres nos da justo eso*: un dataset extra, `more_digits.csv`, con 15.742 muestras nuevas. / El plan fue en dos pasos. Uno, agregar el dataset extra sin tocar el modelo. Si llegábamos al 98 % con eso, listo. / Dos, si no alcanzaba, activar regularización — L2, dropout, augmentation gaussiana, learning rate scheduling — y ver qué combinaciones funcionaban.

0:50

[Slide 27, Ej 3 — el movimiento dominó]

Y este es el resultado del primer paso, que sinceramente nos sorprendió. // Pegamos los dos CSVs antes del split. **Cero cambios al modelo**, solo cambia un campo del config. // Miren los números.

// Validación K-fold=5: subió de 96.22 a 97.06. / Y el test, que es lo que importa: **de 86.30 a 96.36**. // **10 puntos en test, sin tocar el modelo**.

// Y miren acá, en la caja verde: / macro F1 saltó de 0.85 a 0.95. / ¿Se acuerdan que la clase 8 colapsaba en el Ejercicio dos? Ahora la red la predice bien. / El dataset extra cubrió justo el agujero que faltaba. Eso *confirma* la hipótesis.

→ **Mateo→Katia** Le paso a Katia para que muestre la matriz, la tabla y la comparación con el Ej 2.

Bloque H / Katia / Matriz, tabla y comparación visual

/ / 1:45

Foco: mostrar los resultados completos del Ej 3 (matriz, tabla de regularización, comparación con Ej 2).

Q&A: tabla de regularización y técnicas → vos.

[Slide 28, Ej 3 — matriz del ganador] [Bloque H]

Vuelvo después del dominó de Mateo. // Esta es la matriz de confusión del modelo ganador del Ejercicio tres, evaluada sobre el test. / La diagonal está bastante más limpia que la del Ejercicio dos. / La clase 8 que en el dos colapsaba ahora se predice bien, y los errores que quedan se reparten entre varias clases en lugar de juntarse en una sola. / Eso quiere decir que ya no hay un agujero específico de cobertura — las limitaciones que quedan son globales, no de una clase puntual.

0:35

[Slide 29, Ej 3 — resultados (tabla)]

Esta es la tabla completa de los configs que probamos. // Vamos por la columna de test, que es la que vale para nosotros.

Base con datos extra, sin nada de regularización: 96.36. / **Más L2**: 96.56, sumamos 0.20. / **Dropout solo**: peor en test, perdió 0.08. / **L2 más dropout combinados**: peor todavía, perdió 0.32. // Y la fila resaltada en verde, **L2 más augmentation gaussiana con sigma 0.05**: **96.88 %**. 0.52 puntos sobre el baseline. / Ese es nuestro ganador. / Las dos filas siguientes muestran que combinar todo o agrandar la red empeoraba.

1:05

[Slide 30, Ej 3 — comparación visual]

Y este es el plot que compara directamente el Ejercicio dos con el ganador del tres, sobre el mismo test. // Las cinco métricas, lado a lado. La línea roja punteada es el target del 98 %. / Lo que se ve es que la mayor parte del salto vino del dataset extra — ya las barras grandes están al nivel del ganador. La regularización aporta el último empujón.

→ **Katia→Mateo** Mateo vuelve para las curvas y el balance.

Bloque I / **Mateo** / Curvas del ganador + qué ayudó

/ / 2:00

Foco: cierre técnico antes de las conclusiones — curvas del ganador y balance de qué movió la aguja.

Q&A: si te preguntan algo de regularización → Tomás. De dropout en val vs test → Tomás.

[Slide 31, Ej 3 — curvas del modelo ganador] [Bloque I]

Vuelvo para mostrar las curvas y el balance. // Estas son las curvas de aprendizaje del modelo ganador del Ejercicio tres — L2 más augmentation gaussiana, sobre el dataset extendido. / La convergencia es un toque más lenta que el modelo base del Ejercicio dos: *best epoch* alrededor de diez en lugar de cinco, lo cual tiene sentido porque la regularización ralentiza el ajuste. / Pero lo importante es que **train y validación bajan parejas**: no hay overfitting.

0:35

[Slide 32, Ej 3 — qué ayudó y qué no]

Y este slide es el resumen de qué movió la aguja y qué no. // **Lo que ayudó:** / más datos, 10 puntos — de lejos lo más importante. / L2, 0.20 puntos sólidos. / Augmentation gaussiana, 0.32 puntos *adicionales* sobre L2 — las dos juntas suman.

// **Lo que no ayudó:** / Dropout, y este caso es interesante. Dropout gana en validación pero pierde en test. / La razón es que dropout reduce la varianza dentro del train, pero no te sirve cuando el test viene de una población distinta. / La arquitectura *wider*, 200-100 neuronas: mejor en validación, peor en test. Más capacidad memoriza el train mejor pero no generaliza. Y eso *descarta* que el problema sea capacidad. / Y lo último, combinar todo a la vez — L2, dropout y augmentation juntos — nos dio peor que solamente L2 con augmentation. Cuando exagerás la regularización, las técnicas se pelean entre ellas.

→ **Mateo→Katia** Katia cierra con por qué nos quedamos en 96.88 y las conclusiones.

Bloque J / **Katia** / Por qué no 98 % + conclusiones

/ / 2:10

Foco: el cierre que vende la historia. Carisma para el final.

Q&A: si profundizan en qué falta para llegar al 98 % → vos respondés con el argumento del augmentation geométrico.

[Slide 33, Ej 3 — por qué no llegamos al 98] [Bloque J]

Cierro yo. // La pregunta incómoda. ¿Por qué no llegamos al 98 %? Mejor que tenemos: 96.88, faltan 1.12 puntos.

// La hipótesis principal es la siguiente. / El augmentation gaussiano que implementamos simula *ruido píxel a píxel*. Pero la diferencia entre el train y el test no parece ser de ruido — *parecer geométrica*. Distintas personas escriben los dígitos con rotaciones, traslaciones y grosores de trazo distintos. / Y acá viene el detalle importante: *en la clase de regularización vimos cuatro formas de augmentation*: ruido gaussiano, rotaciones, traslaciones y cambios de escala. **Solo implementamos la primera.** / Las otras tres son justo las que atacarían el tipo de variación que falta cubrir en el test.

// Y agrandar el modelo —como mostró Mateo con *wider*— empeoraba. / O sea que **no es un problema de modelo, es un problema de cobertura**. // El siguiente paso natural es implementar augmentation geométrica. Quedó como trabajo a futuro.

1:10

[Slide 34, Conclusiones del TP]

Y para terminar, tres ideas para llevarse del TP. // **Una:** una buena pipeline rinde más que afinar hiperparámetros. El más 10 puntos del Ejercicio tres vino de *datos*, no de modelo. // **Dos:** los empates técnicos son más comunes que los wins claros. Hay que elegir por simplicidad — mirar tiempo y memoria, no obsesionarse con una décima en validación. // **Tres:** “mejor en validación” no es “mejor en test” cuando la población cambia. Dropout fue el ejemplo de manual.

// Tabla de status: / Ej uno cumplido, Ej dos cumplido, Ej tres cumplido salvo el target del 98 %. Mejor obtenido: 96.88. // Lo dejamos documentado con una hipótesis verificable de cómo seguir.

[Slide 35, Gracias] Gracias. / Preguntas.

Apéndice — preguntas anticipadas y a quién derivar**¿Por qué eligieron threshold 0.89? (Katia)**

Porque maximiza F1 en el sweep. Si el cliente prefiere precisión sobre recall, threshold mayor o igual a 0.95. Es decisión de negocio sobre el costo asimétrico, no del modelo.

¿Cuántas muestras tiene el dataset de fraude? ¿Cuál es la prevalencia? (Katia)

Aproximadamente 9 mil muestras totales. La clase positiva pesa alrededor del 12 %. Por eso la curva PR es más informativa que la ROC para este problema.

¿Qué es Adam? ¿No se ve en otra clase? (Tomás)

Adam combina dos ideas: *momentum* (promedio del gradiente, para acumular dirección) y *RMSProp* (learning rate adaptativo por parámetro, según el promedio del gradiente al cuadrado). *El PDF de optimizadores de la cátedra lo recomienda* con los defaults del paper original: $\alpha = 10^{-3}$, $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$.

¿Por qué He para ReLU y Xavier para tanh? (Tomás)

La varianza de las activaciones depende de la activación. ReLU descarta la mitad del input —todo lo negativo se va a cero—, así que para mantener la varianza balanceada por capa hay que multiplicar el desvío por raíz de dos. Xavier asume activaciones simétricas alrededor de cero (tanh, sigmoide).

¿Por qué dropout no funcionó en el Ejercicio tres? (Tomás)

Dropout reduce la varianza dentro de la distribución del train. Acá el problema no es varianza dentro del train: es *cambio de población* entre train y test. Dropout puede ayudar en validación K-fold, donde las muestras vienen de la misma distribución, pero no transfiere a un test que viene de una población distinta.

¿Por qué no llegan al 98 %? (Katia, con apoyo de Tomás)

El techo viene de la calidad del augmentation, no de la capacidad. *La clase de regularización menciona cuatro formas:* ruido gaussiano, rotaciones, traslaciones, cambios de escala. Solo implementamos la primera. Las otras tres son justo las que atacarían el cambio que sospechamos en el test — estilos de escritura. Es follow-up explícito.

¿Cómo eligieron la cantidad de capas y neuronas? (Nicolás, con apoyo de Tomás)

Por la curva U que vimos en la clase de regularización: capacidad muy baja subajusta, capacidad muy alta sobreajusta. Hicimos un sweep alrededor del ganador y elegimos por parsimonia: cuando dos arquitecturas rinden igual, gana la más simple y rápida. *Wider* confirmó la curva U: más capacidad sin más datos $\Rightarrow$ peor en test.

¿Qué relación hay entre lo que hicieron y la clase de regularización? (Tomás)

La clase enseña cuatro técnicas: *early stopping* (slide 14), *augmentation* en cuatro formas (slide 18), *L2/weight decay* (slides 20-25) y *dropout* (slide 26 mencionado). Nosotros usamos las cuatro: early stopping en todo Ejercicio dos y tres; L2 con $\lambda = 10^{-4}$; augmentation gaussiana con $\sigma = 0,05$; y dropout probado pero descartado porque no transfirió. / Lo que faltó: las otras tres formas de augmentation del slide 18.

¿Cómo eligieron el tamaño de batch? (Nicolás)

Sweep en Fase uno con cuatro tamaños: 16, 32, 64, 128. Ganó 16: mismo val_acc que 64 pero *best epoch* en 3 versus 7 y total más rápido en wall clock. El gradiente más ruidoso lo lleva a converger antes.

Fin del guion. Slides en docs/slides_presentacion/slides_presentacion.pdf. Backing teórico para Q&A: docs/notas/ (relu, cross_entropy, early_stopping, decisiones_y_backing_teorico, explicacion_distribution_shift, clase_regularizacion_cosas_implementadas).